

Computational Geometry - Final Project Guidelines

Final Project: Ideas & Proposal Guideline

To help inspire you and clarify the expected scope, we have prepared a list of example project ideas. You are welcome to choose one of these ideas or use them as a starting point for your own unique concept. After deciding on a project, you must formalize your plan in a **Project Proposal**, which is your first deliverable.

Example Project Ideas

These projects are comparable in scope and difficulty. Each one provides an opportunity to tackle a classic problem in computational geometry.

1. Pathfinding Through 2D Polygon Obstacles

- **Description:** Create a tool that finds the shortest collision-free path between a start and end point in a 2D environment filled with polygonal obstacles.
- **Key Geometric Concepts:** This project can be approached in two primary ways, each leading to a different *type* of optimal path:
 - **For the *Shortest Path*:** Implement a **Visibility Graph**, where paths “hug” the corners of obstacles. This directly involves concepts from motion planning.
 - **For the *Safest Path*:** Use a **Voronoi Diagram** or **Delaunay Triangulation**, treating obstacle vertices as sites. The path follows the edges of the diagram, which naturally keeps it centered in open spaces, maximizing clearance from obstacles. This is the idea behind the “Delaunay Navigator” example in the syllabus.

2. Boolean Operations on Polygons

- **Description:** Build a 2D vector drawing application that allows users to create complex shapes by applying **union**, **intersection**, and **difference** operations on simpler polygons.
- **Key Geometric Concepts:** This is a classic application of the **sweep-line paradigm**. A successful implementation will require a robust algorithm for handling segment intersections and managing topology, likely using a **Half-Edge Data Structure (DCEL)**.

3. Low-Polygon Art Generator

- **Description:** Develop a program that converts a photograph or raster image into stylized “low-poly” art. The core task is to strategically place points on the image and use them to generate a triangulation.
- **Key Geometric Concepts:** The heart of this project is the construction of a **Delaunay Triangulation** from a point set. It’s a fantastic application of triangulation and its relationship with **Voronoi Diagrams**.

4. Robot Arm Motion Planner in 2D

- **Description:** Visualize and compute the valid movement space for a multi-jointed robotic arm. Given a start and end configuration, your program should find a collision-free path for the arm to follow.
- **Key Geometric Concepts:** This is a direct application of **Configuration Spaces (C-Spaces)**. The primary challenge is to represent the high-dimensional C-space of the arm and plan a path through it, avoiding obstacles.

5. Alpha Shape Reconstruction from 2D Point Clouds

- **Description:** Implement an algorithm to reconstruct a “tight” boundary shape from a set of unorganized 2D points. Unlike a convex hull, an alpha shape can be concave and can capture more intricate details of the point set’s structure.
- **Key Geometric Concepts:** This advanced topic builds directly on **Delaunay Triangulation** and introduces concepts from **Computational Topology**, such as **simplicial complexes**, to define the shape. It’s a great bridge between pure geometry and shape analysis.

6. Image Vectorizer with Curve Fitting

- **Description:** Create a tool to convert a raster image (like a PNG) into a scalable vector graphic (like an SVG). The process involves tracing pixel outlines, reducing the number of vertices in the resulting paths, and fitting smooth **Bézier curves** to the simplified geometry.
- **Key Geometric Concepts:** This project combines polyline simplification algorithms (e.g., Ramer-Douglas-Peucker) with the practical application of **Bézier curves** and the **de Casteljau algorithm** for creating aesthetically pleasing, smooth shapes.

7. Interactive Map with Geometric Simplification

- **Description:** Build a simple map viewer that can efficiently render complex features like coastlines or country borders at various zoom levels. The core task is to implement a **Level of Detail (LOD)** system by simplifying the geometry of polygons when the user zooms out.
- **Key Geometric Concepts:** The main challenge is to implement a robust **polyline simplification algorithm**. This is a practical and important application in com-

puter graphics and GIS for managing performance without sacrificing essential visual information.

Proposing Your Own Project

We strongly encourage you to be creative! You are free to propose your own project idea. A good original project should:

1. **Have a Clear Geometric Core:** The central challenge must be solving a problem using algorithms and data structures from this course.
2. **Be of Appropriate Scope:** The project should be substantial enough to demonstrate mastery but achievable within the semester timeline.
3. **Be Exciting to You:** Choose something you are passionate about! You will be spending a lot of time on it.

Please discuss any original ideas with the instructors as early as possible to ensure they are a good fit.

Guideline for the Project Proposal (Deliverable 1)

The project proposal is a **1 page document** that outlines your plan for the final project. It is due by **Mar 11**. Its purpose is to demonstrate that you have a clear vision, have identified the core geometric challenges, and have created a realistic timeline for completion. A strong proposal is the foundation of a successful project and your chance to get valuable feedback before you dive deep into coding.

Your proposal should include the following sections:

1. Project Title and Student Information

- A clear, descriptive title for your project.
- Your Name and Student ID.

2. Project Goal

- **What are you trying to build?** Describe the project from a user's perspective. For example, "I will build a tool that takes a 2D point cloud and generates a 'low-poly' art style image."
- **What is the main objective?** Explain the primary goal. For example, "The program will find the shortest path for a robot through a set of obstacles."

3. Core Geometric Concepts

- **Which specific algorithms will you implement?** Name them. For example: “The core of this project is the implementation of a **randomized incremental algorithm** to construct a **Delaunay Triangulation**”
- **Which data structures will you use?** Be specific. For example: “To manage polygon connectivity for the boolean operations, I will implement a **Half-Edge Data Structure (DCEL)** and use a **sweep-line algorithm** to detect intersections.”
- **Why are these the right tools?** Briefly explain why these concepts are essential for solving your project’s main problem.
- **These can be changed later. No need to be too serious**

4. Scope & Key Features (The “Finish Line”)

- **Minimum Viable Product (MVP):** What is the absolute minimum set of features your project must have to be considered complete?
 - *Example:* “1. Load a point set from a file. 2. Compute and display the Delaunay triangulation. 3. Assign a solid color to each triangle based on an underlying image.”
- **Stretch Goals (Optional):** What features would you like to add if you have extra time?
 - *Example:* “1. Allow interactive placement of points. 2. Implement a gradient coloring scheme for the triangles. 3. Export the result as an SVG file.”

Good luck, and we look forward to seeing what you build!